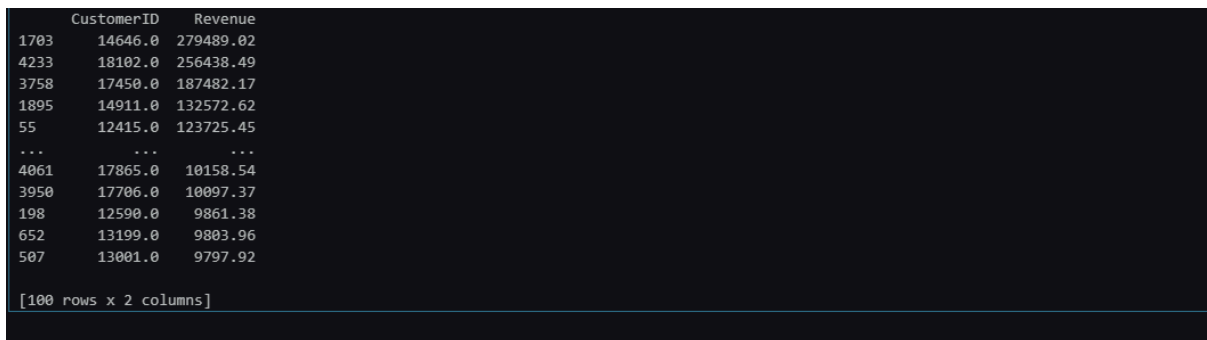


EDA пользователей

1. users.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 1504 entries, 0 to 1503
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   user_id         1504 non-null   int64
1   group           1503 non-null   str
2   age             1504 non-null   int64
3   city            1503 non-null   str
4   device_type     1504 non-null   str
dtypes: int64(2), str(3)
memory usage: 58.9 KB
```

2. Кол-во пропусков

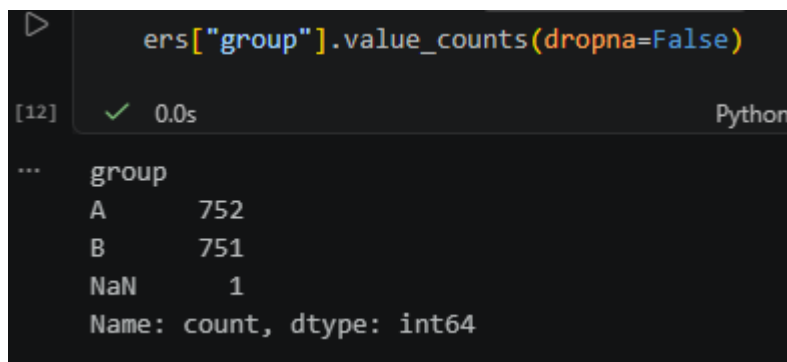


The screenshot shows a Jupyter Notebook cell with a sample of data from the 'users' DataFrame. The data is displayed as a table with columns 'CustomerID' and 'Revenue'. The first 10 rows are visible, showing various values and some missing values (NaN). The output is truncated with '[100 rows x 2 columns]'.

	CustomerID	Revenue
1703	14646.0	279489.02
4233	18102.0	256438.49
3758	17450.0	187482.17
1895	14911.0	132572.62
55	12415.0	123725.45
...	...	...
4061	17865.0	10158.54
3950	17706.0	10097.37
198	12590.0	9861.38
652	13199.0	9803.96
507	13001.0	9797.92

[100 rows x 2 columns]

3. Распределение групп (видно что разделение практически 50/50) группы равные



The screenshot shows a Jupyter Notebook cell with the following code and output:

```
ers["group"].value_counts(dropna=False)
```

[12] ✓ 0.0s Python

```
... group
A      752
B      751
NaN     1
Name: count, dtype: int64
```

4. Анализ возраста

```
valid_age = users[users["age"].between(18, 100)]

print(valid_age.groupby("group")["age"].agg(["count", "mean", "median"]))
```

[14] ✓ 0.0s

	count	mean	median
group			
A	751	32.740346	32.0
B	750	32.654667	33.0

5. Проверка платежей

```
<class 'pandas.DataFrame'>
RangeIndex: 2661 entries, 0 to 2660
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   payment_id            2661 non-null   int64
1   user_id               2660 non-null   float64
2   group                 2661 non-null   str
3   step1_opened          2661 non-null   datetime64[us]
4   step2_entered         2256 non-null   datetime64[us]
5   step3_confirmed       1719 non-null   datetime64[us]
6   step4_success         1522 non-null   datetime64[us]
dtypes: datetime64[us](4), float64(1), int64(1), str(1)
memory usage: 145.7 KB
```

6. Также проверяем пропуски (тут можно увидеть что пользователи выпадают на шагах, но также видно, что в одной строке нет id пользователя, что является критическим пропуском)

```
payment_id      0
user_id         1
group           0
step1_opened    0
step2_entered   405
step3_confirmed 942
step4_success   1139
dtype: int64
```

Поэтому я принял решение удалить данную запись

```
payments = payments[payments["user_id"].notna()].copy()
payments["user_id"] = payments["user_id"].astype(int)
```

Следом проверка, что все платежи относятся к реальным пользователям

```
payments["user_id"].isin(users["user_id"]).value_counts()
```

```
user_id
True      2660
Name: count, dtype: int64
```

7. Корректность последовательности воронки

```
print(
    ((payments["step2_entered"].notna()) &
     (payments["step1_opened"].isna())) .sum()
)

print(
    ((payments["step3_confirmed"].notna()) &
     (payments["step2_entered"].isna())) .sum()
)

print(
    ((payments["step4_success"].notna()) &
     (payments["step3_confirmed"].isna())) .sum()
)
```

```
0
0
0
```

временная последовательность корректная

8. Смотрим сколько было уникальных попыток

```
payments["user_id"].nunique()
```

получаем 1501, то есть из 2661 попытки оплатить, оплат было только 1501. также стоит обратить внимание на кол-во попыток на пользователя.

```
count      1501.000000
mean         1.772152
std          0.901878
min          1.000000
25%          1.000000
50%          2.000000
75%          2.000000
max          4.000000
dtype: float64
```

видно, что максимальное значение 4.

Для теста желательно рассматривать именно завершённые платежи пользователей.

9. Создаем воронку пользователей и считаем ее

```
valid_users = users[users["group"].isin(["A", "B"])].copy()

payments_ab = payments[
    payments["user_id"].isin(valid_users["user_id"])
].copy()

user_funnel = payments_ab.groupby("user_id").agg(
    opened=("step1_opened", "count"),
    entered=("step2_entered", lambda x: x.notna().any()),
    confirmed=("step3_confirmed", lambda x: x.notna().any()),
    success=("step4_success", lambda x: x.notna().any())
).reset_index()

user_funnel = user_funnel.merge(
    valid_users[["user_id", "group", "age", "city", "device_type"]],
    on="user_id",
    how="left"
)

funnel = user_funnel.groupby("group").agg(
    users=("user_id", "nunique"),
    opened=("opened", "count"),
    entered=("entered", "sum"),
    confirmed=("confirmed", "sum"),
    success=("success", "sum")
)

funnel["conv_1_2"] = funnel["entered"] / funnel["opened"]
funnel["conv_2_3"] = funnel["confirmed"] / funnel["entered"]
funnel["conv_3_4"] = funnel["success"] / funnel["confirmed"]
funnel["conv_1_4"] = funnel["success"] / funnel["opened"]

print(funnel)
```

...	users	opened	entered	confirmed	success	conv_1_2	conv_2_3	\
group								
A	751	751	672	549	494	0.894807	0.816964	
B	750	750	707	644	603	0.942667	0.910891	
		conv_3_4	conv_1_4					
group								
A		0.899818	0.65779					
B		0.936335	0.80400					

получается:

A = 65,70%

B = 80,40%

B - тестовая группа

можно увидеть прирост в 22.2%. получается, что из 100 пользователей новая версия приводит к 80 оплатам, тогда как старая к 66.

Также стоит обратить внимание на конверсию по шагам:

1. 1-2 шаг

A = 89.5%

B = 94.3%

прирост на 4.8 пункта

2. 2-3 шаг

A = 81.7%

B = 91.1%

прирост на 9.4 пункта

3. 3-4 шаг

A = 90%

B = 93.6%

прирост на 3.6 пункта

Из этого можно сделать аккуратный вывод, что гипотеза с автозаполнением реквизитов пользователя дает результат, так как на последнем шаге можно увидеть прирост.

Визуализация

1. конверсия

```
import matplotlib.pyplot as plt

conversion = funnel["conv_1_4"] * 100

plt.figure(figsize=(7, 5))
plt.bar(conversion.index, conversion.values)

plt.title("Конверсия из открытия формы в успешную оплату")
plt.xlabel("Группа")
plt.ylabel("Конверсия, %")

for i, value in enumerate(conversion.values):
    plt.text(i, value + 1, f"{value:.1f}%", ha="center")

plt.show()
```



2. воронка целиком

```
import numpy as np

steps = ["Открыл", "Ввел реквизиты", "Подтвердил", "Успешно"]

A = [
    funnel.loc["A", "opened"],
    funnel.loc["A", "entered"],
    funnel.loc["A", "confirmed"],
    funnel.loc["A", "success"]
]

B = [
    funnel.loc["B", "opened"],
    funnel.loc["B", "entered"],
    funnel.loc["B", "confirmed"],
    funnel.loc["B", "success"]
]

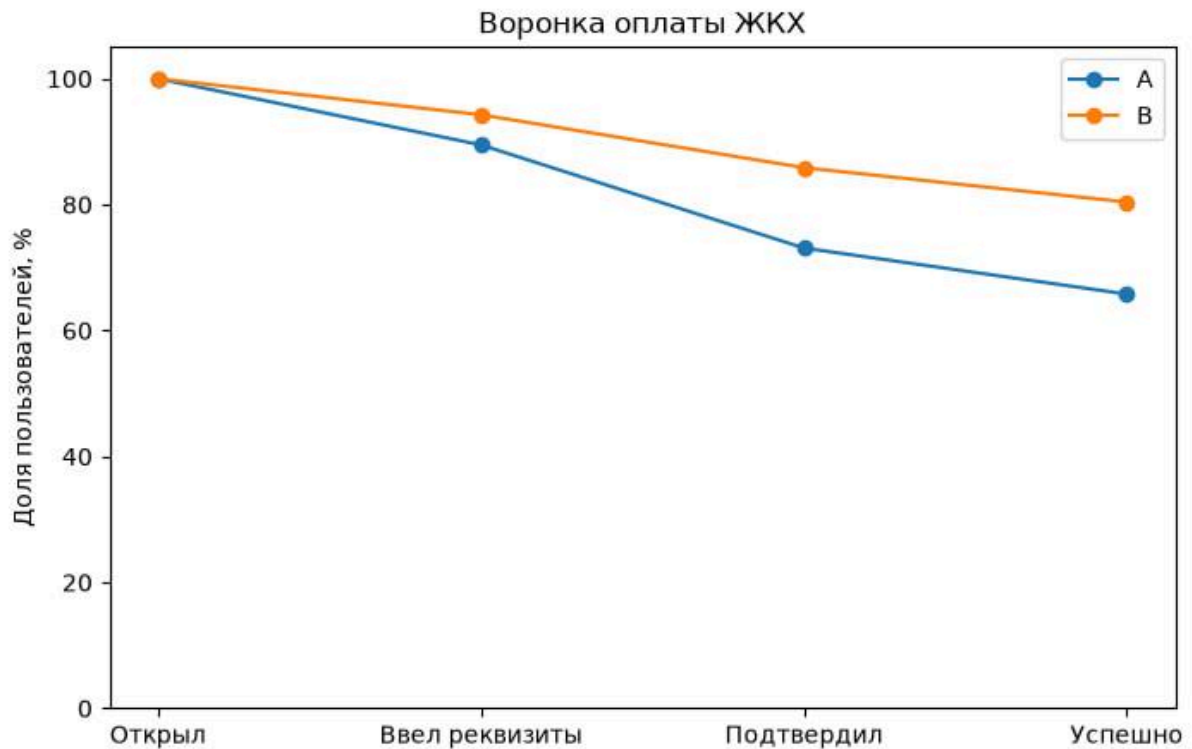
A = np.array(A) / A[0] * 100
B = np.array(B) / B[0] * 100

plt.figure(figsize=(8, 5))

plt.plot(steps, A, marker="o", label="A")
plt.plot(steps, B, marker="o", label="B")

plt.title("Воронка оплаты ЖКХ")
plt.ylabel("Доля пользователей, %")
plt.ylim(0, 105)
plt.legend()

plt.show()
```



10. Статистическая проверка

сравниваем две независимые группы пользователей

обозначу:

K_0 - конверсия одинаковая

K_1 - конверсия в группе B больше конверсии группы A

уровень значимости = 0.05

```
from scipy import stats

p_A = funnel.loc["A", "conv_1_4"]
p_B = funnel.loc["B", "conv_1_4"]

n_A = funnel.loc["A", "opened"]
n_B = funnel.loc["B", "opened"]

diff = p_B - p_A

p_pool = (
    funnel.loc["A", "success"] +
    funnel.loc["B", "success"]
) / (n_A + n_B)
```

```

se = np.sqrt(
    p_pool * (1 - p_pool) *
    (1 / n_A + 1 / n_B)
)

z = diff / se

p_value = 1 - stats.norm.cdf(z)

print("A:", p_A)
print("B:", p_B)
print("Разница:", diff)
print("Z:", z)
print("p-value:", p_value)

```

Результат:

```

A: 0.6577896138482024
B: 0.804
Разница: 0.1462103861517976
Z: 6.385945004473767
p-value: 8.517109240102627e-11

```

наблюдаем разницу в конверсии $65.78 - 80.4 = 14.62$ пункта.

$z = 6.39$. достаточно большое значение и разницу в конверсии нельзя назвать случайной.

$p\text{-value} = 8.5$, учитывая установленный уровень значимости в 0.05, разница также видна

исходя из этого сразу можно отбросить вариант K0 .

Также для наглядности можно проверить uplift = 22.2%, что для продукта является существенным приростом и очень маловероятно, что он случаен.

Доверительный интервал

```
0.10195391016770736
```

```
0.19046686213588784
```

```
0.102
```

```
0.190
```

доверительный интервал в 95%.

В итоге: Новая форма увеличила конверсию. Поставленная гипотеза верна